Level Up Coding ·

Интервью по проектированию систем + Базы данных + Проектирование баз данных +

Концепции проектирования систем + Проектирование систем +

Полное руководство по собеседованию на должность проектировщика баз данных: основные вопросы, которые должен знать каждый инженер

Полное руководство по концепциям проектирования баз данных, передовым практикам разработки и вопросам, которые задают на собеседованиях в ведущих технологических компаниях.

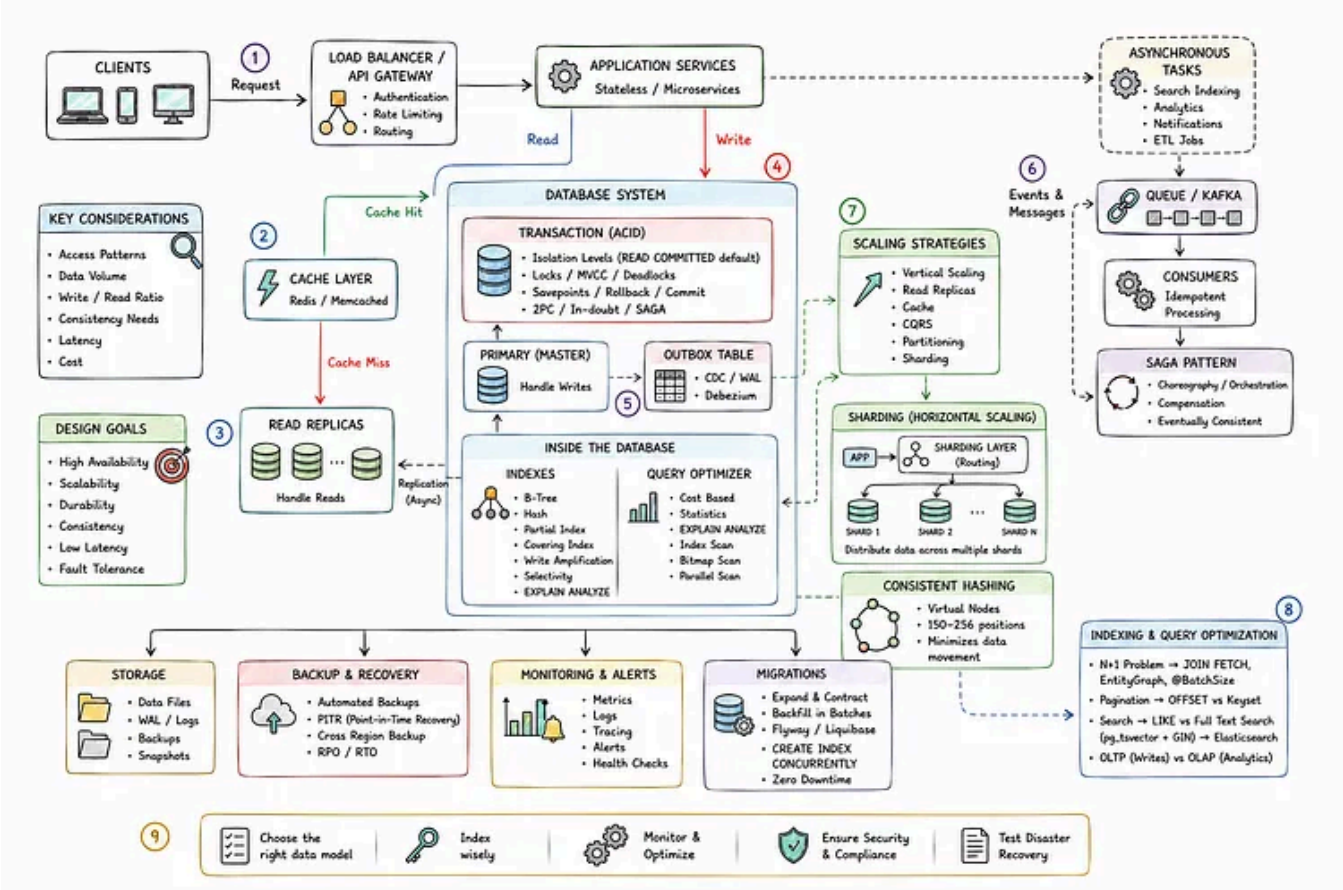
17 мин на чтение · 5 дней назад

V Veenarao

Большинство компаний ожидают, что вы будете знать, как проектировать базу данных и на какие компромиссы идти.

Недавно я участвовал в нескольких собеседованиях в компаниях, специализирующихся на разработке продуктов. Сначала меня спрашивали, какие базы данных я знаю, а потом постепенно стали задавать вопросы о том, какую базу данных я выбрал. Например: почему MongoDB? В каких сценариях вы бы ее использовали? Как работает индексация в MongoDB? и т. д.

Поэтому будьте уверены в том, что знаете о базе данных, а не просто заучивайте ответы. Вы потерпите неудачу, когда вам начнут задавать вопросы, связанные с реальной работой.



Процесс проектирования системы баз данных

Я обобщил все возможные темы, которые могут задать на собеседовании, связанные с проектированием баз данных. Это поможет вам успешно пройти большинство собеседований по проектированию баз данных.

1. SQL или NoSQL:

Четыре решающих фактора при выборе базы данных

- **Модели доступа:** Если вашему приложению нужны гибкие нерегламентированные запросы, лучше использовать SQL. Если запросы предсказуемы и их много, можно оптимизировать схемы NoSQL под них.
- **Масштабируемость:** SQL хорошо подходит для большинства приложений. Выбирайте NoSQL, если вам нужно масштабное горизонтальное масштабирование и миллионы операций чтения или записи в секунду.
- **Согласованность:** Такие приложения, как банковские, платежные, складские и системы бронирования, требуют строгой согласованности. Социальные сети, лайки и аналитика могут допускать несогласованность.

- **Схема:** Используйте SQL для структурированных и стабильных данных. Используйте NoSQL, если схема часто меняется или в записях есть разные поля.

> ***Общее правило:*** *проектируйте базу данных с учетом особенностей доступа к данным в вашем приложении и требований к согласованности, а не с учетом трендов.*

ACID и BASE — что они на самом деле означают в продакшене

ACID гарантирует надежность транзакций за счет **атомарности, согласованности, изолированности и надежности**, что делает ее идеальной для платежей, банковских операций, управления запасами и заказами, где критически важна корректность данных.

BASE (**Basically Available, Soft State, Eventual Consistency**) ставит во главу угла доступность и масштабируемость, допуская временные несоответствия. Поэтому этот подход часто используется в таких системах, как социальные сети, где небольшая задержка в подсчете лайков не является проблемой.

> ***Общее правило:*** *используйте **ACID**, когда важна корректность. Используйте **BASE**, когда временные несоответствия допустимы в обмен на лучшую масштабируемость.*

Теорема CAP: реальный компромисс

Теорема CAP гласит, что во время разделения сети распределенная система может гарантировать только **два** из следующих условий:

- **Consistency** — Everyone sees the latest data.
- **Availability** — Every request receives a response.
- **Partition Tolerance** — The system continues to operate despite network failures.

Since **network partitions are inevitable**, modern distributed systems must support **Partition Tolerance**. The real design decision is choosing between **Consistency (CP)** and **Availability (AP)** based on your application's requirements.

> ***Rule of thumb:*** *Choose **CP** when correctness is more important than uptime, and **AP** when availability matters more than immediate consistency.*

2.Indexing — The Fastest Way to Improve Query Performance

An **index** is a data structure that helps the database find rows quickly without scanning the entire table. Without an index, the database performs a **full table scan**, checking every row.

At a few hundred rows, that's fine. At **millions of rows**, it can become a serious performance bottleneck.

B-Tree vs Hash Index

Most relational databases use **B-tree indexes** by default because they support both **exact lookups** and **range queries**.

- **B-tree:** Best for `=`, `<`, `>`, `BETWEEN`, `ORDER BY`, and prefix searches.
- **Hash:** Optimized for exact matches only. It doesn't support sorting or range queries.

> ***Rule of thumb:*** *Use* ***B-tree*** *indexes unless your workload consists almost entirely of exact equality lookups.*

Composite Indexes

A composite index contains multiple columns.

For an index on **(user_id, created_at)**:

✅ `WHERE user_id = ?`

✅ `WHERE user_id = ? AND created_at > ?`

❌ `WHERE created_at > ?`

This is known as the **left-prefix rule** — the database can only use the index starting from the leftmost column.

> ***Rule of thumb: Rule of thumb:*** *Put columns used in equality (`=`) filters first, followed by columns used for range (`<`, `>`, `BETWEEN`) filters or sorting.*

Covering Indexes

A covering index contains all the columns a query needs, allowing PostgreSQL to perform an **Index Only Scan** without reading the table whenever possible.

This often results in an **Index Only Scan**, making it one of the fastest read optimizations available.

When Indexes Become a Problem
Indexes speed up reads, but they also make **writes more expensive** because every `INSERT`, `UPDATE`, and `DELETE` must update each index.

Another common mistake is indexing **low-cardinality columns**, such as boolean flags or status fields with only a few values. In many cases, the database will ignore these indexes because a table scan is cheaper.

> ***Rule of thumb:*** *Index columns that are frequently used in* ***WHERE, JOIN,*** *and* ***ORDER BY*** *clauses and have high selectivity.*

UUID vs Auto-Increment

Auto-increment IDs are fast because new rows are inserted sequentially. Random UUIDs can cause index fragmentation and slower writes.

Type	Best For	Consideration
Auto-increment	Single database	Fast inserts, but difficult to scale across multiple databases
UUID v4	Distributed systems	Globally unique but causes index fragmentation due to random inserts
UUID v7 / ULID	Modern distributed systems	Globally unique while preserving insertion order, reducing fragmentation

Auto increment vs UUID v4 vs UUID v7

> ***Rule of thumb: Rule of thumb:*** *For modern distributed applications, prefer* ***UUID v7*** *or* ***ULID****. They provide globally unique IDs while preserving insertion order, reducing the fragmentation caused by random UUIDs.*

Partial Indexes
A **partial index** stores only the rows that match a condition, making the index smaller, faster, and cheaper to maintain. A common use case is indexing only **active** records in tables that use **soft deletes.**

> ***Rule of thumb:*** *Index only the data your application queries most frequently.*

3. Transactions & Concurrency

A transaction groups multiple database operations into a single unit of work. Either all operations succeed, or all of them are rolled back. But not all transactions provide the same guarantees — and choosing the wrong isolation level causes subtle, hard-to-reproduce bugs in production.

Isolation levels define how transactions can see and interact with each other's changes.

Isolation Level	Prevents
Read Uncommitted	Nothing (rarely used)
Read Committed	Dirty Reads *(PostgreSQL default)*
Repeatable Read	Dirty & Non-repeatable Reads *(MySQL default)*
Serializable	Dirty, Non-repeatable & Phantom Reads

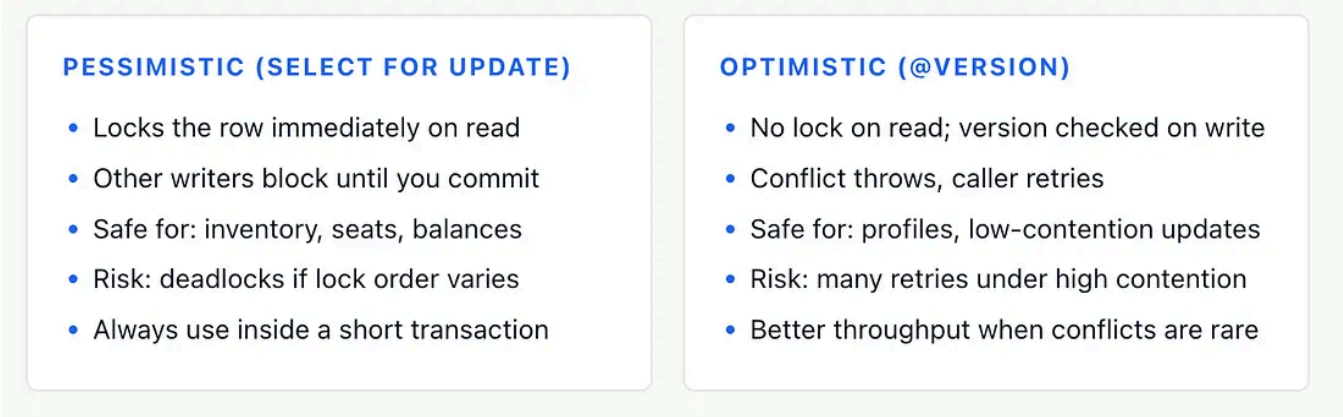
Isolation levels

- **Dirty Read:** Reading uncommitted data.
- **Non-repeatable Read:** Reading the same row twice and getting different values.
- **Phantom Read:** Running the same query twice and seeing extra or missing rows.

> ***Rule of thumb:*** *PostgreSQL defaults to* ***READ COMMITTED****. Each query sees only committed data and gets a fresh snapshot when it starts. This is the right default for most applications. Use Serializable only when correctness is critical (financial calculations, booking systems).*

Pessimistic vs Optimistic Locking

Two common approaches to handling concurrent updates:



Pessimistic vs Optimistic

Pessimistic Locking : Locks the row before it can be updated, preventing other transactions from modifying it until the current transaction finishes.
Best for: Payments, inventory, seat reservations.

Optimistic Locking : Allows concurrent updates and detects conflicts using a version number. If another transaction has already updated the row, the transaction fails and is retried.
Best for: User profiles and applications with low write contention.

```
-- Pessimistic: lock the row immediately
SELECT balance FROM accounts
WHERE id = :id FOR UPDATE;

-- Now safe to modify — no other writer can touch this row
UPDATE accounts SET balance = balance - :amount WHERE id = :id;
```

> ***Rule of thumb:*** *Use* ***Pessimistic Locking*** *when conflicts are common. Use* ***Optimistic Locking*** *when conflicts are rare for better performance.*

Deadlocks

A **deadlock** occurs when two transactions each wait for the other to release a lock, so neither can continue.

The simplest way to avoid deadlocks is to **always acquire locks in the same order** across your application.

> ***Rule of thumb:*** *Prevention is better than recovery — consistent lock ordering avoids most deadlocks.*

2PC vs SAGA
When a transaction spans multiple services, a single database transaction is no longer enough.

- **Two-Phase Commit (2PC)** ensures every participating service either commits or rolls back together. It provides strong consistency but can reduce availability if the coordinator fails..
- **SAGA** breaks a business process into multiple local transactions. If one step fails, compensating actions undo the earlier steps. It scales well but relies on eventual consistency.

> ***Rule of thumb:*** *Use 2PC for tightly coupled systems requiring strong consistency. For microservices,* ***SAGA****is usually the preferred approach because it scales better and avoids distributed locks.*

4. Distributed Patterns: Outbox, CDC, and SAGA

Microservices introduce a fundamental problem: you need to write to your own database *and* publish an event to a message broker — atomically. If your service crashes between the two, your data is saved but the event is lost. Downstream services never learn the update happened.

The Dual-Write Problem

```
// This looks safe. It is not.
BEGIN;
INSERT INTO orders (id, status) VALUES (:id, 'PLACED');
COMMIT;                                  // ← DB write succeeds
kafka.publish('order.placed', event); // ← service crashes here
                                          // Event never sent. Downstream out of s
```

The database and Kafka are different systems. There is no two-phase commit that spans both. You cannot make both atomic with a simple try/catch.

The Transactional Outbox Pattern
Write the event to an `outbox` table in the *same* database transaction as your business data. A separate relay process reads the outbox and publishes to Kafka. If the relay crashes, it restarts and publishes again (at-least-once delivery).

```
BEGIN;
-- Business write
INSERT INTO orders (id, status) VALUES (:id, 'PLACED');

-- Event stored in the SAME transaction
INSERT INTO outbox (aggregate_type, payload, created_at)
  VALUES ('ORDER', '{"id":"...","status":"PLACED"}', NOW());
COMMIT;
-- If this transaction commits, the event WILL eventually be published.
-- If it rolls back, neither write happens. Dual-write problem solved.
```

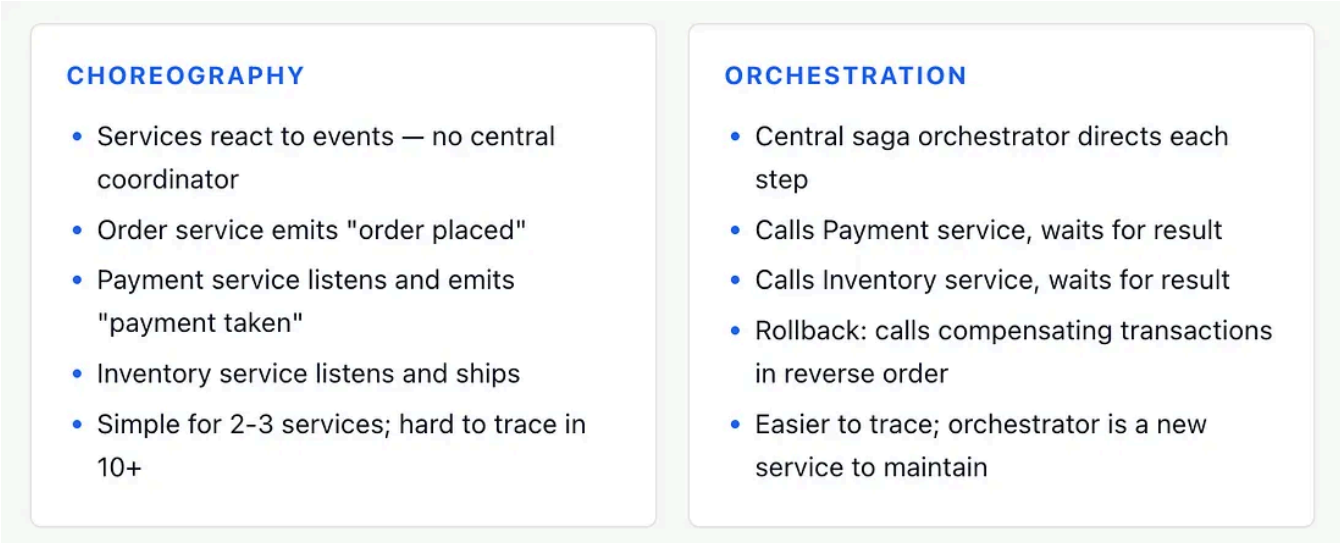
CDC and Debezium: Event-Sourcing from the WAL

Change Data Capture (CDC) reads directly from the database's Write-Ahead Log — the low-level log of every committed change. No outbox table needed; the WAL is the source of truth for every insert, update, and delete that ever happened.

Debezium is the most common CDC tool. It tails PostgreSQL's logical replication stream and publishes change events to Kafka. Downstream services subscribe to those events. Zero code changes to your application — the database itself is the event stream.

When to use what:
Outbox: you control the event schema and payload; you want business events (not raw row changes).
CDC/Debezium: you want database-level change events with no app code changes; good for replication and audit logs.

SAGA: Choreography vs Orchestration



Idempotent Consumers: Handling At-Least-Once Delivery
Message brokers guarantee **at-least-once delivery** — your consumer will receive each message ***at least*** **once**, possibly more if the broker retries. Your handler must be idempotent: processing the same message twice produces the same result as processing it once.

The standard pattern: store a `processed_event_id` table. On each message, try to insert the event ID. If the insert fails with a unique violation — you've already processed it. Skip and acknowledge.

> *At-least-once delivery + a non-idempotent consumer can lead to duplicate charges, duplicate orders, and incorrect inventory counts. Every consumer that handles financial or state-changing events must be idempotent.*

5. Scaling:

Most engineers jump to sharding the moment they hear “scale.” Sharding is the last step, not the first. Here’s the correct order — and why skipping steps is almost always a mistake:

Most applications don’t need sharding. Scale in stages, and only move to the next step when the current one is no longer enough.

1. **Scale up** — Add CPU and RAM. It’s the simplest and cheapest way to improve performance.
2. **Optimize queries** — Add the right indexes, eliminate N+1 queries, and tune connection pooling. This often provides the biggest performance gains.
3. **Add read replicas** — Offload read traffic to replicas while keeping all writes on the primary.
4. **Add a cache** — Cache frequently accessed data with Redis to reduce database load.
5. **Adopt CQRS** — Separate read and write models so each can scale independently.
6. **Shard** — Split data across multiple databases only when a single database can no longer handle the workload.

> **Rule of thumb:** Sharding is the last scaling technique, not the first.

Read Replicas and the Replication Lag Problem
Read replicas improve read scalability, but they introduce **replication lag**. After writing to the primary database, replicas may take milliseconds or even seconds to receive the latest changes.

A common issue is the **read-your-own-write** problem: a user updates their profile, immediately refreshes the page, and sees stale data because the request was served by a lagging replica.

Fix: Route reads to the primary for a short period after a write (sticky routing or session consistency), then switch back to replicas.

Connection Pooling: The Invisible Bottleneck
Creating a database connection is expensive. If every application thread opens its own connection, the database can run out of available connections long before CPU or memory becomes the bottleneck.

Tools like **PgBouncer** reuse a small pool of database connections across many application requests, improving scalability and reducing resource usage.

A common symptom of poor connection management is **"connection pool exhausted"** while the database itself is mostly idle.

CQRS: Separating Read and Write Concerns
Command Query Responsibility Segregation (CQRS) separates the write model from the read model.

- **Write model:** Normalized PostgreSQL optimized for ACID transactions.
- **Read model:** Denormalized stores such as Elasticsearch or Redis optimized for specific query patterns.

Changes are typically propagated using **CDC** or events.

Trade-off: Better read scalability at the cost of eventual consistency and added operational complexity.

Sharding Strategies

When a single database is no longer enough, data can be distributed using one of these sharding strategies.

STRATEGY	HOW IT WORKS	GOOD	BAD
Range	user_id 1–1M → shard 1, 1M–2M → shard 2	Simple; range queries work	Hot shards when traffic skews
Hash	hash(user_id) % N → shard	Even distribution	Range queries hit every shard
Geo	EU users → Frankfurt, US users → Virginia	Low latency; data residency compliance	Cross-region queries are slow

Sharding strategies

6. Consistent Hashing: Routing Data in Distributed Systems

When data is distributed across multiple servers, the system needs to decide which server stores each piece of data.

A simple approach is `hash(key) % N`, where **N** is the number of servers. The problem is that whenever a server is added or removed, most of the keys are reassigned, causing large-scale data movement and cache misses.

Consistent hashing solves this by placing both servers and keys on a virtual ring. Each key is stored on the first server encountered clockwise from its position on the ring. When a server joins or leaves, only a small portion of the keys need to move, making the system much easier to scale.

Virtual Nodes

If each server has only one position on the ring, some servers may end up storing much more data than others.

Virtual nodes solve this by assigning each physical server multiple positions on the ring. This distributes data more evenly and ensures that when a server is added or removed, the data movement is spread across the cluster instead of affecting just one server.

This approach is used by distributed systems such as **Cassandra**, **DynamoDB**, and **Redis Cluster**.

> **Rule of thumb:** Consistent hashing minimizes data movement when the cluster changes, while virtual nodes help balance data evenly across servers.

7. The N+1 Problem: The Silent Performance Killer

The **N+1 problem** is one of the most common performance issues caused by ORMs. It happens when your application executes **one query to fetch a list**, followed by **one additional query for each item** to load related data. What should be a single query turns into hundreds or even thousands of database calls.

```
// 1 query to load orders
List<Order> orders = orderRepo.findAll();
// N queries - one per order, lazily triggered by ORM
for (Order order : orders) {
    String name = order.getCustomer().getName(); // ← hits DB each time
}
// 10 orders in dev → invisible. 10,000 orders in prod → outage.
```

How to Detect It

Common signs of an N+1 problem include:

- Multiple nearly identical SQL queries in the logs (same query, different ID values).
- `hibernate.generate_statistics=true` showing an unexpectedly high number of queries per request.
- APM tools like **Datadog**, **New Relic**, and **Dynatrace** automatically detecting N+1 query patterns.

Common Fixes

- **`JOIN FETCH`** – Load parent and related entities in a single query.
- **`@EntityGraph`** – Define fetch plans declaratively without changing the query.
- **`@BatchSize`** – Fetch related entities in batches instead of one at a time.
- **DTO Projections** — Select only the fields your API needs, avoiding lazy loading altogether.

> **Rule of thumb:** Always index foreign key columns. Without an index (for example, `orders.customer_id`), each of those additional queries may perform a full table scan, making the N+1 problem even worse.

8. Query & Performance: Pagination, OLAP, Full-text Search

Three patterns that commonly cause problems at scale — and the solutions experienced engineers reach for first.

Pagination: Why OFFSET Is Slow and What to Use Instead

`LIMIT ... OFFSET ...` works well for small datasets. However, as the offset grows, the database must scan and discard all preceding rows before returning the requested page. At page 1 it's fast. At page 500 of a million-row table it reads half a million rows and throws them away — `O(n)` per page, getting slower the deeper you go.

Large offsets make pagination increasingly expensive because the database has to skip more and more rows before returning the result.

The fix is **cursor-based (keyset) pagination**:

```
-- Instead of OFFSET, use the last seen ID as a cursor
SELECT * FROM orders
WHERE id > :last_seen_id          -- cursor from previous page
ORDER BY id
LIMIT 20;
-- With an index on id, this is O(log n) regardless of page depth.
-- Returns the last id to the client as the next-page cursor.
```

Keyset pagination is `O(log n)` per page at any depth, and no rows are discarded.

The trade-off: you can only page forward (not jump to page 500), and the sort column must be indexed. For most infinite-scroll feeds and API cursors, this is the right choice.

OLTP vs OLAP: Row-oriented vs Columnar Storage
Row-oriented databases (**Postgres, MySQL**) store complete rows together on disk. This is optimal for transactional workloads: fetching a user's profile needs every column of that one row, so storing them together means one disk read.

Columnar databases (**Redshift, BigQuery, Snowflake**) store each column separately. For an analytics query like `SUM(revenue) GROUP BY region` over 100 million rows, columnar storage reads only the two columns it needs, skips the rest, and compresses each column independently (since column values are often similar). The result: 10–100× faster analytics queries.

DIMENSION	OLTP (ROW-ORIENTED)	OLAP (COLUMNAR)
Optimised for	Many short transactions	Large aggregations over many rows
Query pattern	SELECT * WHERE id = ?	SUM / AVG / GROUP BY over full table
Examples	Postgres, MySQL, DynamoDB	Redshift, BigQuery, Snowflake, ClickHouse
Write pattern	Row-level inserts, updates	Batch loads; updates expensive
Storage	Row blocks	Column segments, compressed

OLTP vs OLAP

A common architecture is to write transactional data to PostgreSQL and replicate it to an OLAP database using CDC or ETL for reporting and analytics.

> ***Rule of thumb:*** *Never run heavy analytics on your production OLTP database.*

Full-Text Search: Why `LIKE` Isn't Enough
Using a query like `WHERE name LIKE '%coffee%'` forces the database to scan every row because the search starts with a wildcard. As your data grows, these searches become slow.

PostgreSQL's **Full-Text Search** solves this by creating an **inverted index** using a `tsvector` column and a **GIN index**. Searches become much faster and also support

features like stemming (e.g., *running* matches *run*), stop words, and relevance ranking.

```
-- Create a tsvector column and GIN index
ALTER TABLE articles
  ADD COLUMN search_vector tsvector
  GENERATED ALWAYS AS (to_tsvector('english', title || ' ' || body)) STORED;
CREATE INDEX idx_articles_fts
ON articles USING GIN (search_vector);
-- Fast, index-based search
SELECT *
FROM articles
WHERE search_vector @@ to_tsquery('english', 'coffee & espresso');
```

If you need features like typo tolerance, autocomplete, advanced relevance tuning, or multilingual search, **Elasticsearch** is a better choice.

APPROACH	WHEN TO USE	COST
LIKE '%term%'	Never in production at scale	Full table scan
pg_tsvector + GIN	Moderate search needs; stay in Postgres	Storage for tsvector column; no new service
Elasticsearch	Complex search; typo tolerance; facets; autocomplete	New service to deploy and monitor

Like vs PostgreSQL full text search vs Elasticsearch

> *OFFSET → Keyset Pagination*
> *OLTP → OLAP for Analytics*
> *`LIKE` → Full-Text Search*
> *Full-Text Search Limitations → Elasticsearch*

9. Zero-downtime Migrations

Schema changes shouldn't take your application offline. On large production tables, a single `ALTER TABLE` can lock writes for minutes, causing an outage. Instead, use the Expand-and-Contract pattern.

The Expand-and-Contract Pattern

1. Expand: Add the new column as nullable and deploy code that writes to both the old and new columns.
2. Backfill: Populate existing rows in small batches using a background job.
3. Validate & Switch: Verify all rows are migrated, add constraints (such as `NOT NULL`), and update the application to read from the new column.
4. Contract: Remove the old column in a later deployment once it's no longer used.

This approach keeps your application online throughout the migration.

> Avoid: Adding a `NOT NULL` column with a default value on a large table in a single migration, as it can rewrite the table and hold long-running locks.

Create INDEX Concurrently
Building an index with a standard `CREATE INDEX` can block writes on large tables. In PostgreSQL, use:

```
-- Safe for production: no locks held
CREATE INDEX CONCURRENTLY idx_orders_user
  ON orders (user_id);
-- Check it completed (won't be used until valid = true)
SELECT indexname, indisvalid
FROM pg_indexes JOIN pg_index ON indexrelid = (
  SELECT oid FROM pg_class WHERE relname = 'idx_orders_user');
```

The index is built in the background, allowing reads and writes to continue with minimal disruption.
Flyway vs Liquibase: Migration Management

	FLYWAY	LIQUIBASE
Format	SQL files (V1__init.sql)	XML / YAML / JSON / SQL changesets
Rollback	Undo SQL (manual / paid tier)	Built-in rollback changesets
Simplicity	Very simple; SQL is the truth	More flexible; more config
Tooling	CLI + Spring Boot integration	CLI + IDE plugins + Spring Boot
Use when	Team knows SQL well; simplicity valued	You need rollback support or multi-DB format abstraction

Flyway vs Liquibase

> *Production Tip: Avoid* `spring.jpa.hibernate.ddl-auto=update` *in production. Manage every schema change with version-controlled migrations using Flyway or Liquibase to keep your database schema predictable and consistent.*

10. Production Pitfalls Nobody Warns You About

Production issues are rarely caused by complex algorithms — they're usually the result of a few common database mistakes.

Hot Rows

When many users update the same row (such as a like counter or inventory count), transactions queue up and become a bottleneck.

Common solutions include:

- Counter sharding: Split one counter into multiple smaller counters and aggregate the total.
- Redis counters: Handle frequent increments in Redis and periodically sync them to PostgreSQL.

Connection Pool Exhaustion

If your application reports "connection pool exhausted" while PostgreSQL is mostly idle, your database connections are likely being held by long-running transactions.

A common cause is opening a transaction and then making a slow network call (such as an HTTP request) before committing. The connection remains occupied the entire time, preventing other requests from using it.

To diagnose long-running idle transactions:

```
SELECT pid,
       state,
       now() - query_start AS duration,
       query
FROM pg_stat_activity
WHERE state = 'idle in transaction'
  AND query_start < now() - INTERVAL '30 seconds';
```

Fix: Keep transactions as short as possible, never make external API calls inside a transaction, and configure `idle_in_transaction_session_timeout` to automatically terminate idle transactions.

Investigate Before Optimizing
Instead of guessing why a query is slow, use PostgreSQL's built-in tools:

- `pg_stat_statements` identifies the queries consuming the most execution time.
- `EXPLAIN ANALYZE` shows the execution plan and whether PostgreSQL is using indexes efficiently or scanning entire tables.
  Measure first, optimize second.

Soft Deletes
Soft deletes (`deleted_at` instead of `DELETE`) are useful but introduce hidden challenges:

- Always filter out deleted rows (`deleted_at IS NULL`).
- Use partial indexes to avoid index bloat.
- Use partial unique indexes so deleted records don't block new inserts.

Backups
A backup is valuable only if you can successfully restore it.

- `pg_dump` creates portable logical backups.
- `pg_basebackup` creates full physical backups for faster recovery.

- WAL archiving + Point-in-Time Recovery (PITR) lets you restore the database to a specific moment, minimizing data loss.

Finally, define your recovery goals:

- RPO (Recovery Point Objective): How much data loss is acceptable.
- RTO (Recovery Time Objective): How quickly the system must be restored.

> Production Tip: Monitor your database, keep transactions short, test your backups regularly, and always optimize based on measurements — not assumptions.

11. Choosing the Right Database

Database choice

> Rule of Thumb:
> ***PostgreSQL****: Best for most applications.*
> ***NewSQL****: When a single PostgreSQL server (even with read replicas) is no longer enough, but you still need SQL and strong consistency.*
> ***DynamoDB****: When you need massive serverless scale, predictable access patterns, and minimal operational overhead.*Use Prometheus + Grafana for monitoring, TimescaleDB or InfluxDB for product analytics and IoT data, and stick with PostgreSQL unless your workload clearly benefits from a specialized database.

Final Thoughts

Good database design is about making the right trade-offs. There is no perfect solution — every choice has advantages and disadvantages.

Before choosing a database or architecture, ask: **How will the data be used? How much traffic will the system handle? What level of consistency is really needed?**

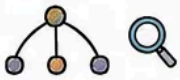
Scan these keywords before your interviews.

DATABASE SYSTEM DESIGN – CHEAT SHEET

SQL vs NoSQL

ACID · BASE · CAP theorem · CP vs AP · Strong / Eventual / Causal consistency · Denormalization · Access patterns · Schema stability · Partition tolerance · Read/write mix

Indexes



B-tree · Hash · O(log n) · O(1) · Left-prefix rule · Covering index · Partial index · Write amplification · Selectivity · EXPLAIN ANALYZE · ANALYZE · UUID v4 · UUID v7 · ULID · Page splits

Transactions

READ UNCOMMITTED · **READ COMMITTED (Postgres default)** · REPEATABLE READ · SERIALIZABLE · Dirty read · Non-repeatable read · Phantom read · SELECT FOR UPDATE · @Version · Deadlock · Lock ordering · Wait-for graph · 2PC · In-doubt transaction · SAGA · Compensating transaction

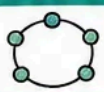
Distributed

Dual-write problem · Outbox table · CDC · Debezium · WAL tailing · At-least-once · Idempotent consumer · SAGA · Choreography · Orchestration · Compensating transaction · processed_event_id

Scaling

Scale up → Tune → Read replicas → Cache → CQRS → Shard · WAL streaming · Replication lag · Read-your-own-write · PgBouncer · Transaction mode · Pool = CPU cores · Range / Hash / Geo sharding · Hot shard · Consistent hashing · Virtual nodes · Ring

Consistent Hashing



Ring · hash(key) mod N is fragile · K/N keys move on resize · Virtual nodes · 150–256 positions per server · DynamoDB partition routing · Cassandra ring · Redis Cluster

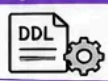
N+1 Problem

Lazy loading · JOIN FETCH · @EntityGraph · @BatchSize · IN(...) · DTO projection · generate_statistics · APM · Index FK columns

Query & Perf

OFFSET = O(n) · Cursor-based · WHERE id > :last · Keyset pagination · OLTP = row-oriented · OLAP = columnar · Redshift · BigQuery · Snowflake · LIKE "%x%" = full scan · pg_tsvector · GIN index · Elasticsearch · BM25 · Inverted index · Fuzzy matching

Migrations



Expand-and-contract · Dual-write · Backfill in batches · Flyway · Liquibase · **ddl-auto: update = never!** · CREATE INDEX CONCRETELY · Invalid index · Federation ≠ sharding

Production

idle-in-transaction · pg_stat_activity · pg_stat_statements · EXPLAIN ANALYZE · Hot row · Counter sharding · Redis INCR · NOT NULL 3-step (nullable → backfill → validate) · Soft delete · Partial index · UNIQUE WHERE deleted_at IS NULL · WAL · pg_dump · PITR · RPO · RTO

DB Choice

Dynamo: serverless · O(1) · GSI · LSI · DAX · hot partition
Mongo: $lookup · aggregation · ad-hoc · Atlas
Cassandra: masterless ring · memtable · SStable · W+R>N
CosmosDB: Azure · multi-API · 5 consistency levels
NewSQL: CockroachDB · PlanetScale · Spanner · HTAP
Time-series: InfluxDB · TimescaleDB · Prometheus

Database cheatsheet

Start with a simple design that meets today's needs, then improve it as your application grows. Understanding these fundamentals will help you build systems that are easier to scale, maintain, and troubleshoot.

Thank you for reading this article. Please provide your valuable suggestions/ feedback.

- *Clap and Share if you liked the content.*
- 📰 *Read more content on my Medium (on Java Developer interview questions)*
- Reach out to me for resume and interview preparation . Contact me on topmate.

Please find my other helpful articles on Java Developer interview questions.

Multithreading and Concurrency Top Interview Questions-Part1

Multithreading and Concurrency Top Interview Questions-Part2

Following are some of the frequently asked Java 8 Interview Questions

Frequently Asked Java Programs

Dear Readers, these are the commonly asked Java programs to check your ability in writing the logic.

SpringBoot Interview Questions | Medium

Rest and Microservices Interview Questions| Medium

Spring Boot tutorial | Medium

Must-know-coding-programs-for-faang-companies| Medium

System Design Interview + Database + Database Design +

System Design Concepts + Design Systems +

Published in Level Up Coding

351K followers · Last published 3 days ago

Coding tutorials and news. The developer homepage gitconnected.com && skilled.dev && levelup.dev

Written by Veenarao

981 followers · 32 following